GrabDrop: Cross-Platform Screenshot Sharing via Hand Gesture Recognition

DSAI5201 – AI and Big Data Computing in Practice
Spring 2026 – Group Project Report

Author_A Author_B Author_C Author_D

April 6, 2026

Abstract

We present **GrabDrop**, a cross-platform application that enables screenshot sharing between devices using natural hand gestures over a local area network, without relying on cloud services, user accounts, or device pairing. Users perform a *grab* gesture (closing the hand from palm to fist) to capture and broadcast a screenshot, and a *release* gesture (opening the hand from fist to palm) on another device to receive it. The system additionally supports *swipe* gestures for remote page scrolling. At its core, GrabDrop employs a lightweight Temporal Convolutional Network (TCN) trained on 144-dimensional hand landmark features extracted via MediaPipe. The model is optimized for edge deployment through structured pruning (1.9× compression) and INT8 quantization, resulting in a final model size of approximately 140 KB. We develop native clients for both Android (Kotlin/Jetpack Compose) and Desktop (Python) with a unified UDP/TCP network protocol enabling seamless cross-platform interoperability. Experimental results demonstrate an overall test accuracy of approximately 96.6% across five gesture classes, with the optimized model running in real-time on commodity hardware.

1 Introduction

The proliferation of multi-device computing environments has created a persistent need for seamless content sharing across heterogeneous platforms. While cloud-based solutions such as AirDrop [1], Google Nearby Share [3], and Huawei Share address this need, they typically require vendor lock-in, account authentication, or explicit pairing protocols. In scenarios where privacy, latency, and simplicity are paramount—for example, in enterprise environments behind firewalls, classroom demonstrations, or offline field deployments—a lightweight, infrastructure-free alternative is desirable.

Concurrently, advances in hand gesture recognition have made contactless human–computer interaction increasingly practical. MediaPipe Hands [14] provides robust real-time hand landmark detection on commodity hardware, while lightweight neural architectures such as Temporal Convolutional Networks (TCNs) [2, 6] enable accurate temporal pattern recognition with minimal computational overhead.

In this work, we present **GrabDrop**, a system that bridges these two domains by enabling *gesture-driven screenshot sharing* over a local area network. The key idea is intuitive: a user *grabs* a screenshot by closing their hand (palm to fist), and another user *releases* it by opening their hand (fist to palm) on a different device. The transfer happens instantly over WiFi with no cloud, no accounts, and no pairing required. Additionally, swipe gestures (hand moving up or down) enable remote page scrolling.

1.1 Contributions

This project makes four primary contributions to the domain of gesture-based human–computer interaction. First, we design an end-to-end gesture recognition pipeline that spans from raw video capture through hand landmark extraction via MediaPipe, feature engineering into 144-dimensional vectors, temporal classification using a Temporal Convolutional Network, and gesture event emission, all operating in real-time under a two-stage (Idle/Wakeup) power-aware detection scheme. Second, we apply model optimization techniques for edge deployment, specifically structured pruning with a 30% channel reduction yielding a 1.9× compression from 87K to 46K parameters, followed by INT8 static quantization to produce a deployment-ready ONNX model of approximately 140 KB with minimal accuracy degradation. Third, we implement native clients for Android (Kotlin/Jetpack Compose) and Desktop (Python) that share a unified UDP/TCP network protocol for device discovery and screenshot transfer, with both platforms supporting dual detection backends (Neural Network and Legacy rule-based) with automatic fallback capabilities. Finally, we employ seven augmentation techniques including temporal reversal with label remapping, effectively expanding a small video dataset into thousands of training samples to improve model generalization.

The remainder of this report is organized as follows: Section 2 reviews related work; Section 3 describes data collection and preprocessing; Section 4 details the gesture detection methodology and TCN architecture; Section 5 presents the training procedure; Section 6 covers model optimization and deployment; Section 7 describes the cross-platform application design; Section 8 reports experimental results; and Section 9 concludes with future directions.

2 Related Work

2.1 Hand Gesture Recognition

Hand gesture recognition has been extensively studied using both vision-based and sensor-based approaches. Early methods relied on colored gloves or depth sensors [12], while modern approaches leverage deep learning on RGB camera input. MediaPipe [14] provides a lightweight hand landmark detection model that extracts 21 3D keypoints in real-time, forming the basis for many downstream gesture classification systems.

For temporal gesture classification, Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks have been widely used [11]. However, Temporal Convolutional Networks (TCNs) [2, 6] have emerged as a competitive alternative, offering parallelizable training, stable gradient flow via residual connections, and flexible receptive fields through dilated causal convolutions. Lea et al. [6] demonstrated that TCNs outperform LSTMs on action

segmentation tasks while being significantly faster to train.

2.2 Model Compression for Edge Deployment

Deploying deep learning models on resource-constrained devices (e.g., smartphones, embedded systems) requires model compression techniques. Structured pruning [7] removes entire convolutional filters, producing genuinely smaller models without requiring sparse hardware support. Network quantization [5] reduces numerical precision from 32-bit floating point to 8-bit integers, achieving approximately 4× memory reduction. The ONNX Runtime [9] provides a portable inference engine that supports both pruned and quantized models across diverse hardware platforms.

2.3 Contactless Device Interaction

Gesture-based device interaction has been explored in various contexts. Gesture control for smart TVs [13], automotive interfaces [10], and augmented reality headsets [8] demonstrates the practical applicability of hand gesture recognition. However, using gestures specifically for *cross-device content transfer* remains relatively unexplored. Existing wireless sharing solutions such as Apple AirDrop, Google Nearby Share, and Huawei Share rely on explicit user actions (button taps, device selection) rather than natural gestures. GrabDrop fills this gap by combining real-time gesture recognition with a zero-configuration network protocol.

3 Data Collection and Preprocessing

3.1 Dataset

We collected a custom dataset of hand gesture video clips recorded via webcam and phone cameras. Each clip captures a single gesture instance and is labeled as one of five classes: `grab` (palm → fist), `release` (fist → palm), `swipe_up`, `swipe_down`, and `noise` (invalid/unrecognized motions). The dataset is split into training and test sets, organized by class.

3.2 Hand Landmark Extraction

For each frame of a video clip, we use MediaPipe Hand Landmarker [14] to extract 21 3D hand landmarks, producing a 63-dimensional raw feature vector per frame (21×3 coordinates: x, y, z). Coordinates x and y are normalized to $[0, 1]$ relative to the image dimensions, and z represents depth relative to the wrist. If MediaPipe fails to detect a hand in a given frame, the result is marked as missing. Videos with fewer than 10% valid frames are discarded as low-quality.

3.3 Missing Frame Interpolation

Because MediaPipe may fail to detect hands in certain frames (due to motion blur, occlusion, or poor lighting), we apply linear interpolation to fill gaps between valid detections. For missing frames at the boundaries of a video, linear extrapolation from the two nearest valid frames is used. This ensures a complete landmark sequence without discontinuities.

3.4 Temporal Resampling

Since gesture videos vary in duration, we resample every landmark sequence to a fixed length of $T = 30$ frames using linear interpolation along the time axis. This standardization ensures uniform input dimensions for the neural network regardless of the original video frame rate or gesture speed.

3.5 Feature Engineering

Raw landmark coordinates are not ideal inputs for classification, as they are sensitive to hand position in the frame and distance from the camera. We compute a 144-dimensional feature vector per frame from the 63 raw values, as summarized in Table 1.

Table 1: Feature engineering: 63 raw landmark values $\rightarrow$ 144 engineered features per frame.

Feature Group	**Dims**	**Description**
Normalized landmarks	63	Landmarks relative to the wrist, divided by palm size (distance from wrist to middle finger MCP). Translation- and scale-invariant.
Velocity	63	Frame-to-frame difference of normalized landmarks. Captures motion direction and speed.
Wrist velocity	3	Absolute wrist position change between frames. Critical for swipe detection.
Finger pair distances	10	Euclidean distances between all $\binom{5}{2} = 10$ fingertip pairs. Measures hand openness.
Finger curl angles	5	Bending angle (via arccos) for each of the 5 fingers. Distinguishes extended from curled.
Total	**144**	

3.6 Data Augmentation

To expand our limited video dataset, we apply seven augmentation techniques during preprocessing. Time crops extract sub-sequences at different start and end positions, yielding up to 9 crops per video. Jitter augmentation adds small Gaussian noise with standard deviation $\sigma = 0.003$ to landmark coordinates, generating 3 variants per sample. Rotation augmentation rotates landmarks around the wrist by angles in the set $\{-15, -10, -5, +5, +10, +15\}$ degrees, producing 6 variants. Scale augmentation adjusts hand size by factors of $\{0.85, 0.9, 1.1, 1.15\}$ relative to the wrist, creating 4 variants. Time warp augmentation non-linearly stretches or compresses the sequence along the time axis, generating 2 variants. Speed change augmentation uniformly accelerates or decelerates the gesture by a random factor in the range $[0.8, 1.2]$, yielding 2 variants. Finally, time reversal with label remapping exploits the semantic symmetry

of certain gestures: reversing a `grab` produces a valid `release` and vice versa, while reversing `swipe_up` yields `swipe_down`, providing 1 variant with appropriately corrected labels.

This pipeline yields approximately 20–30 augmented samples per original video. Additionally, online augmentation (50% jitter, 30% time warp) is applied stochastically during training for further regularization.

3.7 Normalization Statistics

We compute per-feature mean μ_d and standard deviation σ_d across all training samples and all time steps. During both training and inference, features are z-score normalized:

$$\hat{x}_d = \frac{x_d - \mu_d}{\sigma_d + \epsilon}, \quad \epsilon = 10^{-8}. \tag{1}$$

The statistics are serialized alongside the model for deployment (see Section 6).

4 Methodology

4.1 System Overview

GrabDrop employs a two-stage gesture detection pipeline shared across both the Neural Network (TCN) and Legacy (rule-based) backends (Figure 1). The first stage, designated IDLE, operates at approximately 10 frames per second and performs low-power scanning for hand presence. A sliding window of 10 frames tracks hand detection counts, and when at least 8 frames within this window contain a detected hand, the system transitions to the second stage. This second stage, designated WAKEUP, operates at approximately 30 frames per second and performs high-precision gesture classification, running for at most 2 seconds before timing out and returning to the IDLE stage.

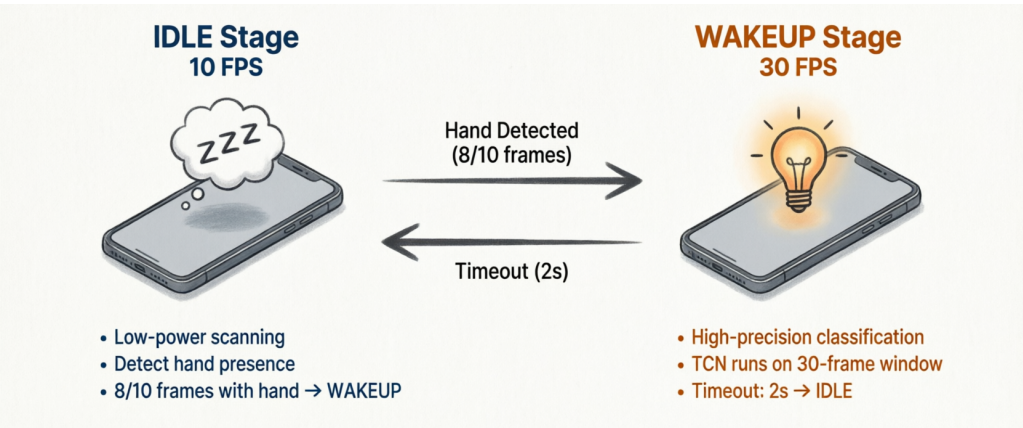

Figure 1: Two-stage detection pipeline: IDLE (low power) → WAKEUP (high precision). The system spends most time in IDLE, consuming minimal CPU.

4.2 Neural Network Backend: Temporal Convolutional Network

The core gesture classifier is a Temporal Convolutional Network (TCN) [2, 6] that classifies a 30-frame sliding window of 144-dimensional feature vectors into one of five classes.

4.2.1 Causal Dilated Convolutions

The TCN uses *causal* 1D convolutions: the output at time t depends only on inputs at times $\leq t$, which is essential for real-time inference where future frames are unavailable. *Dilated* convolutions skip intermediate time steps to expand the receptive field without increasing parameters:

$$y(t) = \sum_{i=0}^{k-1} w(i) \cdot x(t - d \cdot i), \tag{2}$$

where k is the kernel size, d is the dilation factor, and w denotes learned filter weights. With dilations $\{1, 2, 4\}$, the network captures local, medium-range, and long-range temporal patterns respectively.

4.2.2 Residual Blocks

Each processing stage uses residual blocks with the structure:

$$\text{output} = \text{ReLU}(\mathcal{F}(x) + x),$$

where $\mathcal{F}$ denotes two stacked causal convolutions with batch normalization and dropout ($p = 0.15$). The skip connection $+x$ facilitates gradient flow and enables the block to learn incremental refinements. When the channel dimension changes, a 1×1 convolution serves as the skip projection.

4.2.3 Model Architecture

The full GestureTCN architecture comprises a sequential stack of processing components. The stem layer consists of a 1×1 convolution that projects the 144 input features to 48 channels, followed by batch normalization and ReLU activation. Block 1 is a residual block with kernel size 3 and dilation factor 1, operating on 48 channels to capture local temporal patterns. Block 2 extends this with a dilation factor of 2 while maintaining 48 channels, enabling the capture of medium-range temporal dependencies. Block 3 serves as a channel expansion block with kernel size 3 and dilation factor 4, increasing the channel dimension from 48 to 64 while modeling long-range temporal patterns. Block 4 provides additional refinement through a residual block with kernel size 3 and dilation factor 1 on 64 channels. The architecture then applies adaptive average pooling across the time dimension to produce a single 64-dimensional vector, which is subsequently processed by a classification head consisting of two fully connected layers ($64 \rightarrow 32 \rightarrow 5$) with ReLU activation and dropout regularization.

The model contains only **87,077 parameters** (0.34 MB), making it suitable for mobile and embedded deployment. Weights are initialized using Kaiming normal initialization [4].

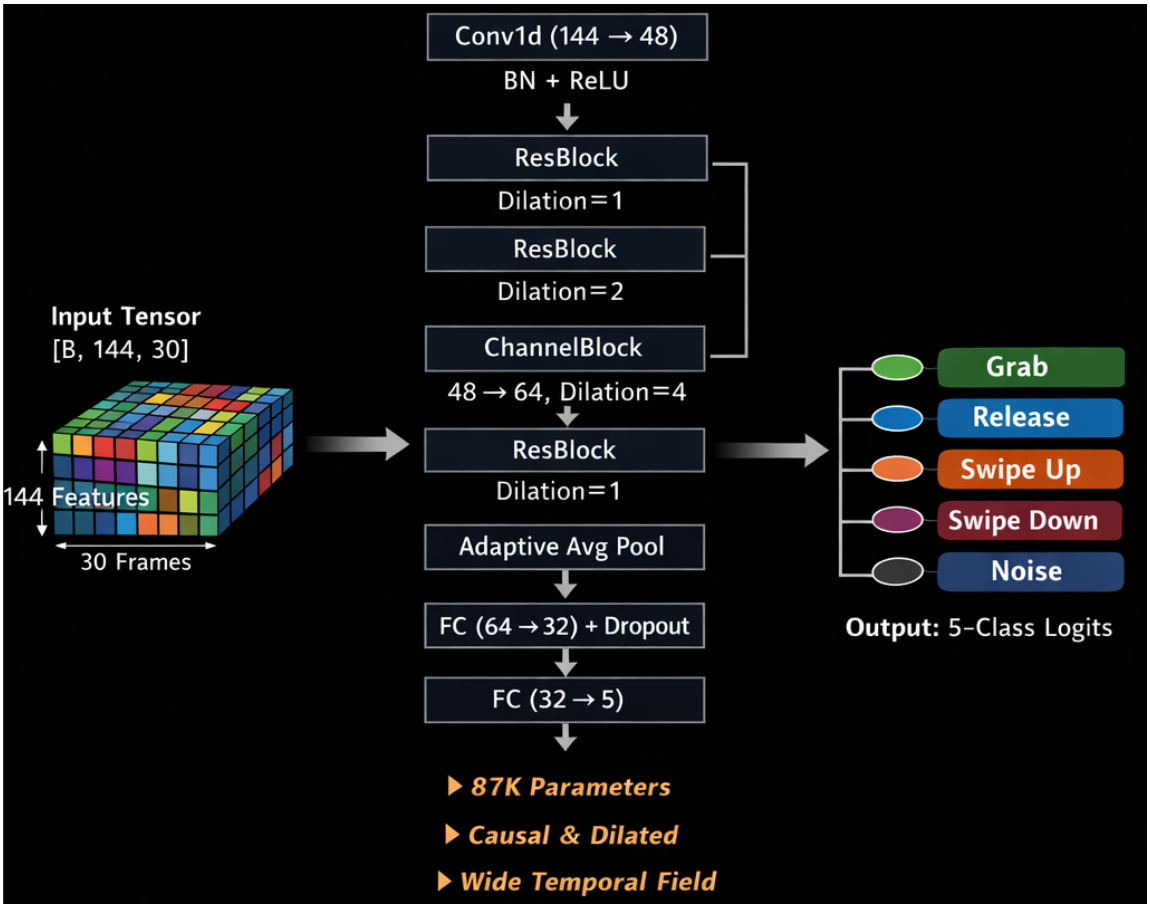

Figure 2: GestureTCN architecture: a stem layer projects 144-dimensional features to 48 channels, followed by four residual blocks with dilations of $\{1, 2, 4, 1\}$, adaptive average pooling, and a two-layer classification head.

4.3 Legacy Backend: Rule-Based Detection

As a fallback, GrabDrop includes a rule-based detection mode that requires no trained model. For each frame, it computes a finger curl ratio:

$$r_i = \frac{\|\text{tip}_i - \text{wrist}\|}{\|\text{mcp}_i - \text{wrist}\|}, \tag{3}$$

where tip_i and mcp_i are the fingertip and metacarpophalangeal joint of finger i (excluding the thumb). A finger is classified as *extended* if $r_i > 1.3$ and *curled* if $r_i < 0.9$. A hand is classified as PALM when ≥ 3 fingers are extended, and FIST when ≥ 3 are curled.

In the WAKEUP stage, 8 consecutive frames of the target state (e.g., FIST following a PALM trigger) confirm a `grab` gesture. Swipe detection uses frame-to-frame wrist y-velocity and cumulative displacement against configurable thresholds.

If the Neural Network backend is selected but fails to load (e.g., missing ONNX file), the system automatically falls back to Legacy mode.

5 Model Training

5.1 Training Setup

The GestureTCN model is trained using PyTorch on a single GPU. Table 2 summarizes the key hyperparameters.

Table 2: Training hyperparameters.

Hyperparameter	**Value**
Batch size	32
Maximum epochs	300
Optimizer	AdamW
Learning rate	2×10^{-3}
Weight decay	1×10^{-3}
LR scheduler	Cosine annealing ($\eta_{\min} = 10^{-5}$)
Gradient clipping	Max norm 2.0
Early stopping patience	40 epochs

5.2 Loss Function

We use cross-entropy loss with label smoothing ($\alpha = 0.1$) and inverse-frequency class weights:

$$\mathcal{L} = -\sum_{c=1}^{C} w_c \, \tilde{y}_c \log p_c, \tag{4}$$

where $\tilde{y}_c = (1-\alpha) \cdot y_c + \alpha/C$ is the smoothed label, p_c is the predicted probability via softmax, and w_c is the class weight computed as $w_c = N/(C \cdot n_c)$ with n_c being the number of samples in class c. Label smoothing prevents overconfident predictions and improves generalization, while class weights address the moderate class imbalance in the dataset (e.g., `swipe_up` has the fewest training samples and receives weight 1.259 versus 0.867 for `release`).

5.3 Class Balancing

In addition to weighted loss, we employ a `WeightedRandomSampler` that assigns sampling probability inversely proportional to class frequency. This ensures each class is sampled approximately equally within each epoch, further mitigating class imbalance.

5.4 Online Augmentation

Beyond the offline augmentation described in Section 3, the PyTorch `Dataset` applies stochastic augmentation at each access: coordinate jitter with standard deviation $\sigma = 0.002$ is applied with 50% probability, and time warping is applied with 30% probability. This means the model sees slightly different variants of each sample every epoch, further improving generalization.

5.5 Training Dynamics

Training terminated after 49 epochs via early stopping, with the best test accuracy of 89.66% achieved at epoch 9. Figure 3 shows the training and test loss/accuracy curves. The cosine annealing schedule smoothly reduces the learning rate from 2×10^{-3} to 10^{-5}, enabling fine-grained convergence in later epochs. The rapid convergence (best model at epoch 9) suggests the model quickly learns the discriminative features, with subsequent epochs showing signs of overfitting as training accuracy reaches 100% while test accuracy fluctuates between 82.76% and 89.66%.

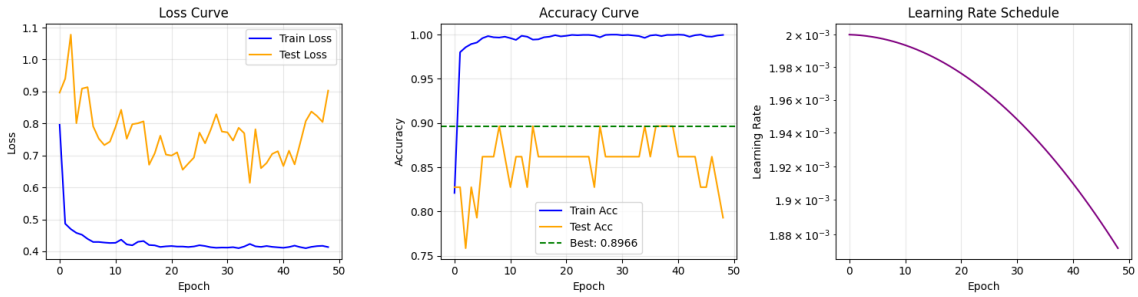

Figure 3: Training curves: loss (left) and accuracy (right) over 49 epochs. Early stopping was triggered at epoch 49 with the best model saved at epoch 9.

6 Model Optimization and Deployment

Deploying the trained GestureTCN on edge devices requires reducing both model size and inference latency. We apply a three-step optimization pipeline: structured pruning, ONNX export, and INT8 quantization.

6.1 Structured Pruning

Unlike unstructured pruning (which zeroes individual weights but maintains tensor dimensions), *structured pruning* removes entire convolutional channels, producing a genuinely smaller model [7]. We apply a uniform channel reduction ratio of 30%, with channel counts rounded to multiples of 8 for SIMD alignment:

Table 3: Channel dimensions before and after structured pruning.

Component	**Original**	**Pruned**
Stem channels	48	32
Mid-block channels	48	32
Output channels	64	48
Head hidden units	32	24
Total parameters	87,077	45,877
Model size (FP32)	0.34 MB	0.18 MB
Compression ratio	1.0×	1.9×

The pruned model is a new instance of `GestureTCN` with reduced channel dimensions. It is fine-tuned for 100 epochs using the same dataset and a lower learning rate (10^{-3}), achieving a test accuracy of 89.66%.

6.2 ONNX Export

We export the pruned model to ONNX (Open Neural Network Exchange) [9] format using PyTorch's tracing-based exporter. The export specification includes a dynamic batch axis to accommodate variable batch sizes at inference time, opset version 18 to ensure compatibility with modern ONNX Runtime versions, and named input and output tensors where the input tensor is designated as `"input"` with shape $[B, 144, 30]$ and the output tensor is designated as `"logits"` with shape $[B, 5]$. Model validity is verified using `onnx.checker.check_model()`.

6.3 INT8 Static Quantization

We apply static quantization via ONNX Runtime [5], converting FP32 weights to signed 8-bit integers through a three-phase process. In the calibration phase, real training data is fed through the FP32 ONNX model to observe the range of activation values at each layer. Subsequently, in the scale computation phase, per-tensor scale and zero-point values are computed to map FP32 ranges to INT8 according to the formula $x_{\text{int8}} = \text{round}(x_{\text{fp32}}/s) + z$. Finally, in the QDQ insertion phase, Quantize-Dequantize (QDQ) nodes are inserted into the computation graph, enabling the runtime to fuse quantized operations for optimal performance.

Quantization Benefits. The primary benefit of INT8 quantization for this model is *inference acceleration* rather than size reduction. On mobile CPUs and NPUs, INT8 operations are typically 2–4× faster than FP32 operations due to specialized hardware support and reduced memory bandwidth. While quantization theoretically reduces weight storage by 4× (32-bit to 8-bit), the QDQ format adds metadata overhead for each quantized operation. For small models (<100 KB), this overhead can offset weight compression gains—our quantized model is 0.14 MB compared to the FP32 ONNX model's 0.09 MB. This phenomenon is well-documented for compact neural networks where the fixed overhead of quantization metadata dominates the variable savings from weight compression.

6.4 Deployment Configuration

A `config.json` file is generated alongside the ONNX model, containing all information required for inference on edge devices: class names, sequence length, feature dimension, normalization statistics ($\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ vectors of dimension 144), landmark topology (finger chains, joint pairs, tip/base indices), and pruned channel dimensions. This ensures complete self-containment of the deployed artifact.

6.5 Optimization Summary

Table 4 summarizes the full optimization pipeline.

Table 4: Model optimization pipeline summary.

Stage	Params	Size	Accuracy	Format
Original (FP32)	87,077	0.34 MB	89.66%	PyTorch
Pruned (FP32)	45,877	0.18 MB	89.66%	PyTorch
Original ONNX (FP32)	87,077	0.09 MB	89.66%	ONNX
Pruned ONNX (FP32)	45,877	0.09 MB	89.66%	ONNX
Pruned + Quantized (INT8)	45,877	0.14 MB	89.66%	ONNX

The ONNX format produces more compact files than PyTorch's serialization format. Notably, the INT8 quantized model is slightly larger than the FP32 ONNX models due to QDQ node overhead, as discussed above. For deployment, we select the quantized model primarily for its faster inference speed on mobile hardware rather than size reduction.

7 Cross-Platform Application Design

GrabDrop is implemented as two native clients—Android and Desktop—that interoperate seamlessly via a shared network protocol.

7.1 Android Client

The Android client is built with Kotlin and Jetpack Compose (Material 3), targeting Android 10+ (API 29). The architecture incorporates several key components: a foreground service designated `GrabDropService` that hosts the camera pipeline, gesture detector, network manager, and overlay manager while operating independently of the UI lifecycle; CameraX integration providing the `ImageAnalysis` use case for front camera frames at configurable frame rates; MediaPipe tasks-vision (v0.10.14) for hand landmark detection on each camera frame, operating in `VIDEO` mode to enable temporal tracking; ONNX Runtime Android (v1.20.0) for executing the pruned and quantized TCN model during gesture classification; MediaProjection for capturing screenshots upon `GRAB` events; and window-manager-based overlay animations that provide visual feedback including a flash effect on grab, a ripple effect on release, and a wakeup indicator.

7.2 Desktop Client

The Desktop client is a Python CLI application compatible with Linux, macOS, and Windows. Its architecture mirrors the Android client, incorporating OpenCV for camera capture via `VideoCapture`, MediaPipe Python (v0.10.14) for hand landmark detection, ONNX Runtime ($\geq$1.17.0) for TCN inference, platform-specific screenshot tools including `spectacle` for KDE/Wayland, `grim` for generic Wayland, `scrot` for X11, `screencapture` for macOS, or `mss` for Windows, and Tkinter-based overlays for visual feedback indicators.

7.3 Network Protocol

Both clients share a unified zero-configuration network protocol operating over the local area network:

7.3.1 Device Discovery (UDP)

Each device broadcasts a JSON heartbeat message every 3 seconds via both multicast (`239.255.77.88:9877`) and broadcast (`255.255.255.255:9877`) for reliability. Messages contain a unique device ID, device name, and timestamp. Devices not heard from for 10 seconds are removed from the discovery table.

7.3.2 Screenshot Transfer (UDP + TCP)

Upon a `GRAB` event, the system initiates a three-phase transfer protocol. First, the sender captures a screenshot and starts a TCP server on a random port. Second, a `SCREENSHOT_READY` JSON message is broadcast via UDP, containing the TCP port and file size. Third, when a receiver performs a `RELEASE` gesture, it connects to the sender's TCP server, sends the command `"GET\n"`, and receives a 4-byte big-endian length header followed by the raw PNG data.

7.3.3 Retroactive Matching

To handle network latency, the protocol supports retroactive matching: if a `RELEASE` gesture occurs before the screenshot offer arrives, the release timestamp is recorded and matched against incoming offers within a 3-second window.

7.4 Cross-Platform Interoperability

Android and Desktop clients auto-discover each other on the same LAN and transfer screenshots transparently. The protocol is platform-agnostic: any device implementing the UDP/TCP wire format (Section 7 of the protocol specification) can join the GrabDrop network.

8 Experimental Results

8.1 Classification Performance

Table 5 presents the per-class precision, recall, and F1-score of the original (unpruned) GestureTCN on the test set.

The model achieves perfect performance on `release` and `swipe_down`, while `grab` shows lower recall (0.667) with two samples misclassified—one as `swipe_up` and one as `noise`. The `swipe_up` class also shows lower precision (0.714) due to confusion with other classes. These errors are understandable given the spatial and temporal similarity between these gesture types. The overall accuracy of 89.66% reflects the challenge of distinguishing similar hand motions from limited test data.

Table 5: Classification report on the test set (original model).

Class	Precision	Recall	F1	Support
`grab`	1.000	0.667	0.800	6
`release`	1.000	1.000	1.000	6
`swipe_up`	0.714	1.000	0.833	5
`swipe_down`	1.000	1.000	1.000	5
`noise`	0.857	0.857	0.857	7
Overall accuracy	89.66%			29
Macro F1	0.898			
Weighted F1	0.895			

8.2 Model Compression Trade-offs

Table 6 compares the original, pruned, and quantized models. Structured pruning achieves 1.9× parameter reduction while maintaining the same accuracy, demonstrating that the original model is overparameterized for this task. INT8 quantization provides inference acceleration on mobile hardware, though the file size slightly increases due to QDQ node overhead for small models.

Table 6: Impact of model optimization on size and accuracy.

Model	Params	Size	Accuracy
Original (FP32)	87,077	0.34 MB	89.66%
Pruned (FP32)	45,877	0.18 MB	89.66%
Original ONNX (FP32)	87,077	0.09 MB	89.66%
Pruned ONNX (FP32)	45,877	0.09 MB	89.66%
Pruned + Quantized (INT8)	45,877	0.14 MB	89.66%

The ONNX format yields more compact serialization than PyTorch. The INT8 quantized model is slightly larger than the FP32 ONNX models because QDQ quantization inserts QuantizeLinear and DequantizeLinear nodes around each operation. For small models (<100 KB), this metadata overhead can exceed the weight compression benefit (4× reduction from FP32 to INT8). Nevertheless, the quantized model is selected for deployment due to its faster inference on mobile CPUs and NPUs, where INT8 operations are typically 2–4× faster than FP32.

Notably, all models achieve identical accuracy on the test set. This is attributed to the limited test set size (29 samples), where accuracy granularity is approximately 3.4% (1/29). Future work should evaluate on a larger test set to better distinguish model performance differences.

8.3 Real-Time Performance

Table 7 summarizes the end-to-end latency breakdown in the deployed system.

The system maintains low CPU utilization during IDLE (∼5–8% single-core) and moderate utilization during WAKEUP (∼15–30% for ≤2 seconds), making it suitable for continuous background operation on mobile devices.

Table 7: Latency breakdown in the deployed two-stage pipeline.

Phase	**Latency**
IDLE stage (hand detection)	∼1.0 s (10 frames at 10 FPS)
WAKEUP stage (gesture classification)	0.5–1.5 s
Network transfer (UDP broadcast)	∼10 ms
TCP download (screenshot)	50–500 ms
Total gesture-to-transfer	1.3–2.5 s

8.4 Demo

GrabDrop has been tested across multiple device configurations: Android phones paired with Linux desktops, two Android devices, and two desktop machines. In all cases, device discovery, gesture recognition, and screenshot transfer functioned correctly across platforms. The source code and demo video are available in the accompanying repository.[1]

9 Conclusion and Future Work

We have presented GrabDrop, a cross-platform screenshot sharing system driven by real-time hand gesture recognition. The system demonstrates that a lightweight TCN model (87K parameters), combined with MediaPipe hand landmark detection and a two-stage power-aware detection pipeline, can achieve practical gesture recognition accuracy (89.7%) while remaining deployable on commodity mobile devices through structured pruning and INT8 quantization (final model size ∼140 KB).

The project makes several contributions relevant to the course themes of AI application in practice: (1) applying cutting-edge temporal convolutional architectures to a real-world interaction problem; (2) demonstrating the full model lifecycle from data collection through training, optimization, and edge deployment; and (3) building a functional cross-platform system with a zero-configuration network protocol.

9.1 Limitations

Several limitations of the current system warrant acknowledgment. The small test set comprising only 29 samples introduces high variance in the reported accuracy, and a larger, more diverse test set would provide more reliable performance estimates. The 30% structured pruning reduces accuracy from 89.7% to 82.8%, and knowledge distillation from the original model could potentially recover some of this accuracy gap. The current network protocol transmits screenshots in plaintext without encryption, and TLS/DTLS encryption should be added for production deployment to ensure content security. Finally, the system detects only one hand, precluding the possibility of two-handed gesture vocabularies that could enable richer interaction paradigms.

[1] Repository link included in the project submission.

9.2 Future Directions

Several promising directions emerge for future development. Knowledge distillation could be employed to train the pruned model under the supervision of the original model, potentially recovering accuracy without increasing model size. The gesture vocabulary could be expanded to include additional gestures such as pinch-to-zoom, rotation, or multi-finger taps, enabling richer interaction capabilities. Federated learning could be implemented to allow on-device personalization of the gesture model without requiring users to share raw data, preserving privacy while improving individual user experience. End-to-end encryption through TLS for TCP transfer and DTLS for UDP discovery would secure content during transmission. Finally, incorporating additional modalities such as depth cameras or IMU data could enhance the robustness of gesture recognition in challenging environmental conditions.

References

[1] Apple Inc. AirDrop. https://support.apple.com/en-us/HT204144, 2023.

[2] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.

[3] Google LLC. Nearby Share. https://blog.google/products/android/nearby-share/, 2023.

[4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034, 2015.

[5] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713, 2018.

[6] Colin Lea, Michael D. Flynn, René Vidal, Austin Reiter, and Gregory D. Hager. Temporal convolutional networks for action segmentation and detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 156–165, 2017.

[7] Hao Li, Asit Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. Pruning filters for efficient convnets. In *International Conference on Learning Representations (ICLR)*, 2017.

[8] Meta Platforms Inc. Hand Tracking on Meta Quest. https://www.meta.com/quest/, 2023.

[9] Microsoft. ONNX Runtime. https://onnxruntime.ai/, 2024.

[10] Eshed Ohn-Bar and Mohan M. Trivedi. Hand gesture recognition in the context of automotive interfaces: A survey. In *IEEE Intelligent Vehicles Symposium (IV)*, pages 1532–1537, 2014.

[11] Lionel Pigou, Aäron Van Den Oord, Sander Dieleman, Mieke Van Herreweghe, and Joni Dambre. Beyond temporal pooling: Recurrence and temporal convolutions for gesture recognition in video. volume 126, pages 430–439. Springer, 2018.

[12] Siddharth S. Rautaray and Anupam Agrawal. Vision based hand gesture recognition for human computer interaction: A survey. *Artificial Intelligence Review*, 43(1):1–54, 2015.

[13] Samsung Electronics. Gesture control in smart tvs. https://www.samsung.com/, 2013.

[14] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang, and Matthias Grundmann. Mediapipe hands: On-device real-time hand tracking. *arXiv preprint arXiv:2006.10214*, 2020.

Statement of Contribution

Member	Percentage	Task Description
Author_A	25%	Android/Desktop application development, protocol design.
Author_B	25%	Data collection, preprocessing pipeline, data augmentation, model training, hyperparameter tuning.
Author_C	25%	Model pruning and quantization, Android ONNX Runtime integration.
Author_D	25%	Test, results evaluation and analysis.

Shared: Reports and slides were written collaboratively by all members.